# *Notes on Information Theory*

*Riccardo Lo Iacono*

April 30, 2026

Document is WIP, typos could be found

**Contents.**


**1 Introduction** **1**

1.1 Information and Shannon's theory . 1

1.2 Encoding a source . . . . . 2

1.3 The Kraft-McMillan inequality . . 4

1.4 Average codeword length . . . . . 5

**2 Huffman encoding** **7**

2.1 Canonical Huffman encoding . . . . 8

2.2 Adaptive Huffman encoding . . . . 10

**3 Arithmetic encoding** **11**

3.1 Adaptive arithmetic encoding . . . 12

**References** **14**

**Acronyms** **14**

# - 1 - Introduction.

At the core of modern computer science, among others, two fields have become fundamental: *information theory* and *data compression*. The former deals on how informations can be transmitted through a channel; the latter with methods to maximize the throughput of the channel. More correctly, data compression studies how to efficiently encode data; that is, how to store informations efficiently.

In what follows, we discuss topics of both fields in parallel; we choosed this approach as the two fields are closely related.

## - 1.1 - Information and Shannon's theory.

Above we've used the term information assuming its meaning as given; it should be clear that the notion of information in the spoken language has little to do with the CS counterpart.

Formaly, the notion of information is that of a measure of the reduction of the uncertainty of the outcomes of an event. That is, a measure of how much knowledge is gained observing the event. Therefore, it makes sense that the information carried by an event is inversely proportional to its probability.

Historically, the first person attempting to define information was R.V.L. Hartley; see [1] for more details. But the major contribution is due to Claude Shannon, who in [3], building on Hartley's ideas, provides a rigorous definition of information and a way to compute it.

In his paper, Shannon shows that any communication system can be reduced to five parts: a *sender*, a *transmitter*, a *communication channel*, a *receiver* and a *destination*. A schematic representation of such system is shown in Figure 1.

With his paper, Shannon introduces introduces two important hypotesis:

- Transmitting a symbol requires a non-zero amount of time;
- The communication system is subject to noise.

**Remark:** In his paper Shannon actually develops two theories: one with noise and one without. We focus on the latter.

The question now is: what properties should a measure of information satisfy? According to Shannon, such a function should have a dependence on probability, should be monotonic, continous and additive for indenpendent events.

More often than not, we are interestend in computing the information a source; that is, an entity over some alphabet $\Sigma$, that at any given time produces a symbol $\sigma \in \Sigma$ with probability $\Pr(\sigma)$. For instance, a Markov chain

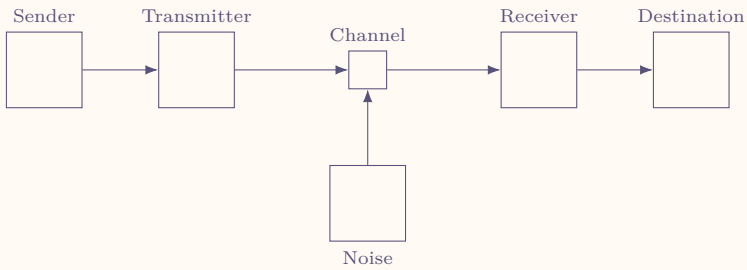



**Figure 1:** Schematic representation of Shannon's communication system.

is an example of source of information. We shall say that there exist two types of sources: with memory and memoryless. In what follows, we consider memoryless sources.

But how do we actually measure information? According to Shannon, the only valid measure is

$$H(S) = K \sum_{i=1}^{n} p_i \log_b p_i \tag{1}$$

where $n$ is the total number of symbols produced by $S, p_i$ the probability associated woth the $i$-th symbol produced and $K$ a positive constant usually set to 1. Unless stated otherwise, the base $b$ of the logarithm is always 2. We usually refer to Equation (1) as *entropy of a source $S$*.

Although we could prove that Equation (1) is the only valid measure, it is beyond our scope; we remind the interested reader to [3][Appendix 2].

## - 1.2 - Encoding a source.

A problem arises with sources: the alphabet these uses are, in general, not suitable for transmission. Is for this reason that, as shown in Figure 1, we make use of transmitters/receivers. Although we describe only the encoding process of a source, the deconding is trivially symmetric.

Let $S$ be a source. A code is defined as a map from the source alphabet to a new alphabet, called input alphabet. In Sections 2 and 3 and **??** we consider some particular family of codes, while in the remainder of this section we introduce some classes of code, as well as some theoretical tools useful in studing codes.

### - 1.2.1 - Non-singular codes.

In the previous section, we've defined a code as a map from the source alphabet to a new one. The question is: are all code equal? It's easely understood that this is not the case; some code are in fact useless to our needs. For instance, if

we define a code that maps two distinct character to the same codeword, the receiver will have a hard time decoding the text. A code whose codewords are all distinct is called *non-singular*.

As we show in the next section, non-singular codes are not enough; and additional properties are needed to make a good code.

### - 1.2.2 - Uniquely decodable codes.

Non-singular codes are not enough. More often than not, it happens that the sequence of bit in analysis may be factored in as concatenation of distinct codewords, one again making the decoding process tedious.

Briefly speaking, what we are looking for are codes for which any sequece of codewords corresponds to at most one sequece of source symbol. We call such code Uniquely Decodable (UD) codes.

Hence the questions: how do we construct UD codes? And how do we determine if a given code is UD? In the present section we answer to the latter question; the answer to the first question will be discussed later.

In 1953, Sardinas and Patterson proved a result that defines an implicit algorithm to determine if a code is UD. The result, stated below, is based on a simple observation: if we try to construct two factorizations, each word in the first factorizations is either a sub-word in the other factorization or has a prefix that is a suffix of the words in the other factorization.

**Theorem (Sardinas-Patterson):** *Let $\mathcal{C}$ be a code over the alphabet $\Sigma$. Define the sets $S_0, S_1, \ldots$ as follow:*

$$S_0 = \mathcal{C}$$
$$S_i = \{w \in \Sigma^* \mid \exists \alpha \in S_0, \exists \beta \in S_{i-1} : \alpha = \beta w \vee \beta = \alpha w\}.$$

*Then, necessary and sufficient condition for $\mathcal{C}$ to be UD is that, for all $n > 0$, $S_0 \cap S_n = \varnothing$.*

Algorithmically speaking, since the number of possible sets it's finite, the process will eventually stop. The condition in the above theorem, defines three possible halt conditions, namely:

1. $\exists i \geq 0$ s.t.$S_0 \cap S_i \neq \varnothing$, meaning the code is not UD;
2. $\exists i \geq 0,$ s.t. $S_i = \varnothing$, making the code UD; and
3. $\exists i, j \geq 0$ s.t. $S_i = S_j$, meaning the code is UD.

In the example following we use colours to show the relatioship between $\alpha, \beta$ and $w$. Additionaly, we align each $w$ to the row of the corresponfing $\alpha/\beta$.

**Example:** Consider the code $\mathcal{C} = \{a, c, ad, abb, bad, deb, bbcde\}$, is it UD? Let's apply the Sardinas-Patterson algorithm and check. What we get is the following table.

| $S_0$ | $S_1$ | $S_2$ | $S_3$ | $S_4$ | $S_5$ |
|---|---|---|---|---|---|
| a | | | | | |
| c | | | de | | |
| ad | d | | | | |
| abb | bb | | | | ab |
| bad | | | | | |
| deb | | eb | | b | |
| bbcde | | cde | | | bcde |

Since $S_0 \cap S_5 = \{ab\} \neq \varnothing$, we can conclude that $\mathcal{C}$ is not UD.

### - 1.2.3 - Prefix codes and the Kraft-McMillan inequality.

One of the most useful properties a code can have is that of being prefix-free; that is, no two codewords are prefix of one another. Thanks to this property, each codeword can be decoded unambigously, i.e. the word is decoded instantaneously. We call codes with this property *prefix-free*, or simply prefix. We also use the term instantaneous interchangably.

For this class of code the following results hold.

**Theorem:** *A code is instantaneous if and only if it's prefix-free.*

**Theorem:** *All prefix-free codes are UD.*

***Proof.*** Follows as a consequence of Sardinas-Patterson theorem. □

## - 1.3 - The Kraft-McMillan inequality.

We now consider an interesting result about UD codes: the Kraft-McMillan inequality. Formally, the Kraft-McMillan theorem stastes that a code $\mathcal{C}$ over an alphabet of size $d$, with of codewords lengths $l_1, \ldots, l_n$ is UD, if and only if

$$\sum_{i=1}^{n} d^{-l_i} \leq 1.$$

The inequality provides therefore a necessary and sufficient condition for a code to be UD.

***Proof.*** We prove the result by construction; that is, we show that if a set of lengths satisfies the inequality, then we can build a tree whose paths from the root to the leaves define codewords of a prefix-free code.

$\Longleftarrow$ Easely seen with any UD code.

$\Longrightarrow$ Let $l_1, \ldots, l_n$ be such that

$$\sum_{i=1}^{n} d^{-l_i} \leq 1.$$

Denote by $n_j$ the number of codewords of length $j$. Let $L$ be the maximum length. Then

$$\sum_{j=1}^{L} n_j = n$$

since there are $n$ codewords. Hence, we have

$$\sum_{i=1}^{n} d^{-l_i} = \sum_{j=1}^{L} n_j d^{-j} \leq 1.$$

By doing so, we build a $d$-ary tree which has $n_1$ leaves at depth 1, $n_2$ leaves at depth 2, etc. It's easely seen that this produces a prefix-free code. $\square$

**Remark:** If the set of lengths is such that the sum is strictly less then one, then some paths in the tree will correspond to no codeword.

## - 1.4 - Average codeword length.

We now turn our attention to a more interesting question: fixed a source over some alphabet, how do we design the optimal code for it? The question, apparently hard, has a somewhat obvious answer: take in account the lengths of the codewords. Or better said, their average.

Formally speaking, we take in account the Average Codeword Length (ACL) of the code, defined as the quantity

$$L_S(\mathcal{C}) = \sum_{i=1}^{n} p_i l_i$$

such that is minimal. We say that a code $\mathcal{C}$ is compact (or optimal) for the source $S$ if, among all prefix codes for $S$, $\mathcal{C}$ has the minimal ACL.

**Remark:** Is there a lower bound for the ACL? Yes, and it given by the following result.

**Lemma (Shannon):** *If $\mathcal{C}$ is a UD code for a source $S$ with probabilities $p_1, \ldots, p_n$, and $d$ is the size of the code alphabet, then*

$$L_S(\mathcal{C}) \geq \frac{H(S)}{\log d}$$

In addition to the ACL, two additional quantities are significant when designing a code: the efficiency $\vartheta_S(\mathcal{C})$ and the redundancy $\rho_S(\mathcal{C})$. More precisely, we want that the efficiency, defined as

$$\vartheta_S(\mathcal{C}) = \frac{H(S)}{L_S(\mathcal{C})}$$

is maximized; while the redundancy, defined as

$$\rho_S(\mathcal{C}) = 1 - \vartheta_S(\mathcal{C}) = \frac{L_S(\mathcal{C}) - H(S)}{L_S(\mathcal{C})}$$

is minimized.

A question still remains: how do we build optimal codes? Over the years, several attempts to provide a procedure to build optimal codes we done, but until 1952 none achived the goal. We discuss this procedure in the next section.

# - 2 - Huffman encoding.

The first method to build optimal codes was discovered by D. A. Huffman, as solution problem posed by his lecturer Robert Fano[1].

Before we discuss such a method, let us observe some properties of optimal codes. The following hold.

1. $\forall i, j, p_i \geq p_j \implies |c_i| \leq |c_j|$. We could otherwise swap the codeword and obtain a code with a lower ACL.

2. The least probable symbols have the same length. It's easely noted that if not, we could find a code with a lower ACL.

3. The longest codewords differ on the last symbols. It follows directly from the previous property.

The procedure itself is pretty straightforward; it builds a $d$-ary tree, $d$ being the size of the code alphabet, label each brach with a different symbols of the code alphabet and assigns the codewords by traversing the tree. Practically speaking, we can summarize the algorithm as follow:

1. If not available, compute the symbols probabilities by scanning the text.

2. Sort the symbols according to their probabilities, non-increseingly.

3. Make sure that the number of source symbols $n$ is $d + \alpha(d-1)$, for some integer $\alpha$. If needed add *dummy* symbols with probability 0.

4. Merge the least $d$ probable symbols in to a node until a root is formed.

5. Assign the codeword by traversing the tree.

Is it obvious that encoding a text with Huffman has a significant overhead depending on the code alphabet. We must infact transmit, in addition to the bit stream, also the tree. Decoding the code is straightforward as well: we simply traverse the tree and decode each symbol.

In terms of complexity Huffman encoding/decoing takes $\mathcal{O}(n \log n)$, where $n$ is the number of source symbols.

In addition to the tree overhead, we have to consider the possibility in which the tree gets corrupted. In the next few sections we consider some variation to the plain Huffman encoding, that try to fix these issues.

[1] At that time, Fano and Shannon were looking for a technique to build optimal codes.

**Example:** Consider the source $S = \{a, b, c, d, e, f, g\}$, where $\Pr(a) = 0.3, \Pr(b) = 0.125$, $\Pr(c) = 0.125, \Pr(d) = 0.05, \Pr(e) = 0.1, \Pr(f) = 0.1, \Pr(g) = 0.2$. Provide a ternary Huffman code for $S$.

To begin with, let us sort the source symbols, we have: $a \succeq g \succeq b \succeq c \succeq e \succeq f \succeq d$. Since the number of symbols is of the form $d + \alpha(d-1) = 3 + 2 * 2$, we can move on with merging the nodes. Thus, we consider the three least probable symbols and we merge them; in our case the symbols $d, e, f$. Iterating the procedure we get the tree in Figure 2.

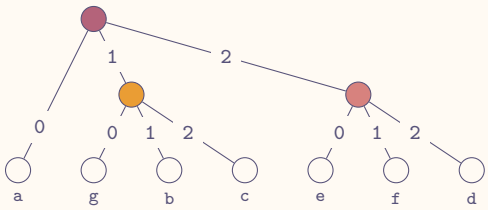



**Figure 2:** Huffman tree for the source $S$.

Therefore, we get the code $\mathcal{C} = \{0, 10, 11, 12, 20, 21, 22\}$.

Assume now receiving the stream 222122111101020; knowing that the tree used is the one above, what is the encoded string? By trivially traversing the tree, we can easely decode the stream as $dfdbbage$.

## - 2.1 - Canonical Huffman encoding.

As stated previously, Huffman decoding needs the tree used in compression to correctly decode the bit-stream; it's obvious that, the larger the source alphabet is, the larger is the outputed tree. Can we avoid transmitting the whole tree then?

In the present section we describe a variation of the classical Huffman encoding that does exactly so. We'll show that all that we need are the symbols to transmit, ordered non-increasingly, and the first codeword for each length.

Before we proceed, let us observe something: there exist optimal codes that correspond to no Huffman tree, for instance the tree on the right in Figure 3, corresponds to no Huffman, yet it's optimal.

Let's now consider how to compute the Canonical Huffman encoding. Starting from the Huffman encoding of the source, compute the length of the codewords and construct the following data structures:

- A num array, to store at num[l] the number of codewords of length $l$.
- A symb array, that at symb[l] stores the list of symbols having codeword length of $l$-bits.

- A fc array, storing at fc[l] the first codeword off all symbols encoded with $l$-bits.

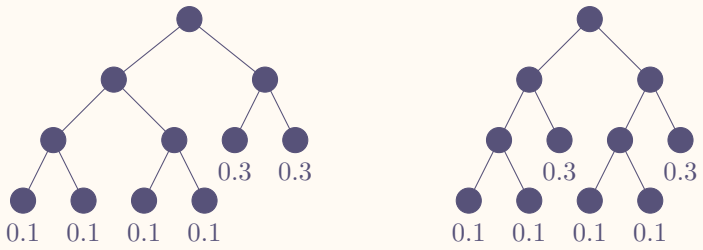



**Figure 3:** Comparison of Huffman tree and an equivalent optimal tree.

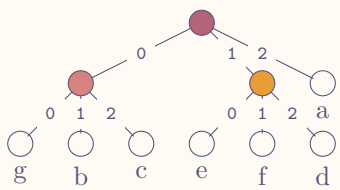
Using the above structures, we assign consecutive codewords to the symbols in symb[l] starting from fc[1]. All the arrays are 1-based.

Building the num and symb array is trivial, the question is how to build fc. Let MAX be the maximal codeword length, then fc[MAX] = 0, as a consequence of the tree we are building, for any $l \leq$ MAX, fc[l] = fc[l + 1] + num[l + 1] / d, where $d$ is the size of the alphabet. The pseudo-code in Procedure buildFC uses a bottom-up approach, hence the tree built is unbalanced to the left.

**Procedure** buildFC

```
fc[MAX] = 0
for l ← MAX − 1 to 1 do
    fc[l] ← (fc[l + 1] + num[l + 1]) << 1
    l ← l − 1
end
```

**Example:** Assume we want to compute the Canonical Huffman encoding for the tree in Figure 2. It follows easely that num = {1, 6} and symb = {a, {g, b, c, e, f, d}}. We now proceed building the fc array. By using the procedure we have fc[2] = 0 while fc[1] = (fc[2] + num[2])/ 3 = 2. The tree we obtain, symmetric to that of Figure 2, is shown below.




To conclude our discussion, we provide the Procedure decodeCanonical-Huffman to decode string encoded using Canonical Huffman. Recall that the

decoder has access to both fc and symb; therefore, we can simply read bits until the value is less than fc[l], for all $l$.

**Procedure** decodeCanonicalHuffman

```
v ← next_bit()
l ← 1
while v < fc[l] do
    v ← 2 * v + next_bit()
    l ← l + 1
end
return symb[l, v - fc[l]]
```

## - 2.2 - Adaptive Huffman encoding.

In the previous section we discussed Canonical Huffman encoding, which solves the issue of transmitting the tree, but not that of scanning the text twice. In this section we shall present a variant of Huffman encoding that solves this issue: the Adaptive Huffman encoding.

Proposed independently by Fallar in 1973 and Gallager in 1978, later improved by Knut, the Adaptive Huffman encoding solves the issue of repeated scan of the text own of the classic approach. The core idea is simple: build a optimal Huffman tree for the text process thus far, and reorganize the tree when necessary.

More in details, we can summarize the procedure as follow:

1. Let $T$ be an empty tree, or let it contain a special symbol NYT.
2. While scanning the text, for each symbol:
    - If it has already occurred, use the available codeword to encode it;
    - If not, encode the NYT symbol, then add the symbol as a new leaf and encode it.
3. After encoding a symbol, update $T$ as follow:
    - Increment the frequency of the symbol and all its ancestors;
    - Reorganize the tree to maintain the Huffman properties.

It can be proved that, in general, the adaptive approach and the classic one have the same performances.

# - 3 - Arithmetic encoding.

In Section 2, we said that Huffman encoding was proposed by D. A. Huffman as answer to Fano's question. Huffman's answer wasn't the only one. In fact another of Fano's student, Peter Elias, suggested a technique which came to be known as Arithmetic encoding.

The core idea is extreamly simple: instead of defining a codeword for each symbol, let's encode the whole text in one go. Precisely, the text is encoded as decimal number in the range [0, 1].

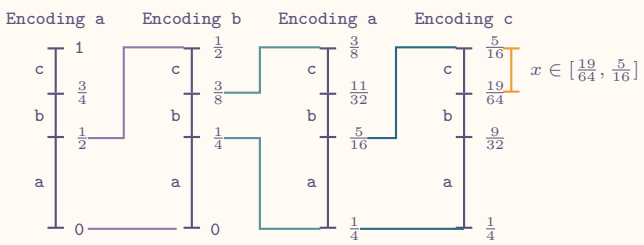
Algorithmically speaking, we proceed as follow: divide the range [0, 1] in to $n$ subintervals, each proportional to the probability of a distinc symbol, which we can assume to be lexicographically sorted; each time a symbol of the input text is read, select the corresponding subintervals and repeat the splitting procedure. Once the whole text is read, output a decimal number with the last interval.

Below we provide the pseudo-code of the procedure just described.

**Procedure** encode(S, n, Prob)

```
l ← 0
s ← 1
for i ← 1 to n do
    s ← s * Prob[S[i]]
    l ← l + s * f_{S[i]}
    /* Assume f_{S[i]} to be the cumulative distribution
       function                                          */
    i ← i + 1
end
return x ∈ [l, l + s]
```

**Example:** Consider the string $S = abac$, how do we encode it? By apply the encoding procedure we get the following.



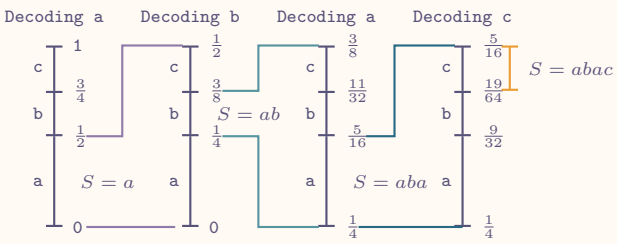

The question now is: how many bits do we need? Some result in number theory, allows us to say that we need excatly $\lceil \log 2/s \rceil$ bits.

The decoding process is symmetric: the range [0, 1] is divided in $n$ subintervals, at each iteration the proper range is selected and the intervals recomputed accordingly. The process is valid because the procedure is syncronized; that is, both encoder and decoder know the source alphabet and the associated probabilities. Below the pseudo-code code of the procedure to decode a string encoded with Arithmetic encoding.

**Procedure** decode(Stream, n, Prob)

```
l ← 0
s ← 1
S ← ""
for i ← 1 to n do
    S ← S + σ /* Assume + to be the concatenation operation
        and σ the read symbol.                              */
    s ← s * Prob[S[i]]
    l ← l + s * f_{S[i]}
    /* Assume f_{S[i]} to be the probability cumulative function
        */
end
return S
```

**Example:** Consider $x = {}^{39}/_{128}$. If the source symbols and probabilities are that of the previous example, what is the decoded string?



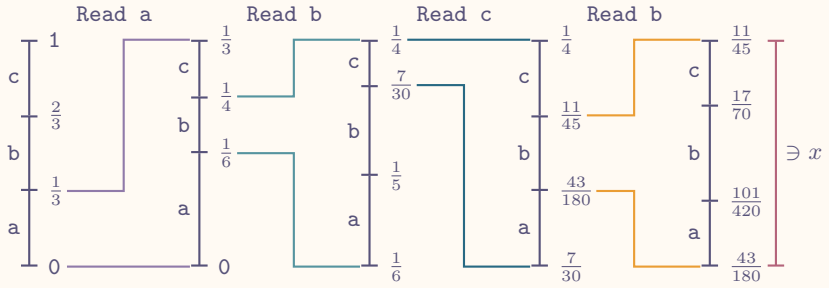

## - 3.1 - Adaptive arithmetic encoding.

As for Huffman encoding, the arithmetic encoding procedure just discussed has an issue: the probabilities needs to be known before the encoding begins. A question then arises: can we do better as for the adaptive Huffman encoding? That is, can we make arithmetic encoding adaptive? In the present section we discuss how we can achive adaptability.

The idea is the same as that of adaptive Huffman, but instead of updating the tree, we update the known probabilities. Algorithmically speaking,

we begin by assuming all symbol have the same probability; that is, the $n$ subintervals have the same width. As before, once a symbol is read we select the corresponding subinterval, and additionally we increase that symbol probability. The selected interval is then divided in $n$ subintervals with width proportional to the probabilities.

**Example:** Let $S = abcb$ be the text to encode. Before any character is read, the probabilities are: $p_a = p_b = p_c = 1/3$. Once the first character ($a$) is read, we select the range $[0, 1/3]$ and update the probabilities as $p_a = 1/2, p_b = p_c = 1/4$. We repeat this process until the end of the text, encoding $S$ as $[43/180, 11/45]$.




The decoding step is simmetrycal: instead of updating the probabilities once a symbol is read, these are updated each time a new one is decoded. Apart from these caveats, both the encoding and decoding procedures are equal to Procedure encode and Procedure decode.

## References.


[1] R. V. L. Hartley. "Transmission of Information". In: *Bell System Technical Journal* 7.3 (1928), pp. 535–563. DOI: 10.1002/j.1538-7305.1928.tb01236.x.

[2] David A. Huffman. "A Method for the Construction of Minimum-Redundancy Codes". In: *Proceedings of the IRE* 40.9 (1952), pp. 1098–1101. DOI: 10.1109/JRPROC.1952.273898.

[3] Claude E. Shannon. "A mathematical theory of communication". In: *Bell Syst. Tech. J.* 27.4 (1948), pp. 623–656. DOI: 10.1002/J.1538-7305.1948.TB00917.X.

## Acronyms.

**ACL** Average Codeword Length. 5

**UD** Uniquely Decodable. 3